4.1.9 Logic Programming

The basic way that *Mathematica* works is to take an expression you give as input, and then generate from it a chain of expressions by applying various transformation rules. At each step, *Mathematica* tries rules in order, using the first one that applies.

Logic programming systems such as Prolog are, like *Mathematica*, based on transformation rules. But, unlike *Mathematica*, such systems emphasize applying many transformation rules in parallel. Instead of producing a single chain of expressions by applying one rule at each step, logic programming systems typically apply many different rules at each step, thereby generating many different chains of possible expressions. Whereas *Mathematica* typically "commits" to a particular rule at each step, logic programming systems try out many different possibilities, committing to a particular one only when they get the result of the whole calculation.

One of the advantages of logic programming is that you do not have to define a particular "path" for a calculation to take. The system tries out all possible paths, keeping only the ones that give desired result. While potentially flexible, this approach has the serious problem, however, that it is impossible to predict in advance even approximately how long any given calculation will take. Depending on exactly what rules have been given, the system could spend a very long time checking out many chains of transformations that do not work out.

Mathematica is designed to be more predictable in its operation. It simply tries the rules it knows, in a particular order, and uses the first one that applies.

There are some circumstances, however, when *Mathematica* does effectively try out several possibilities, much like a logic programming system.

The "test" in this transformation rule prints the forms of `x`, `y` and `z` that *Mathematica* is trying. Since the `Print` does not return `True`, *Mathematica* goes on trying other possibilities.

```
In[1]:= f[x__, y__, z__] := none /; Print[{x}, {y}, {z}]
```

This shows all the possible partitions of arguments that *Mathematica* uses in trying to use the transformation rule for `f`.

```
In[2]:= f[1,2,3,4,5]

{1}{2}{3, 4, 5}
{1}{2, 3}{4, 5}
{1, 2}{3}{4, 5}
{1}{2, 3, 4}{5}
{1, 2}{3, 4}{5}
{1, 2, 3}{4}{5}

Out[2]= f[1, 2, 3, 4, 5]
```

When *Mathematica* is trying to match a pattern, there are sometimes several possible ways that it can fill in blanks. If, for example, the pattern contains several

_ or __, *Mathematica* may have to try several different partitionings of function arguments. Similarly, if the pattern involves an `Orderless` function, *Mathematica* may have to try the arguments in all possible orders. Whether a particular way of filling in blanks works is sometimes determined by consistency with other parts of the pattern, but may also be determined by the results of `/;` conditions.

By giving many rules with different `/;` conditions, you can force *Mathematica* to test out several transformations. In fact, you could in principle implement a full logic programming system in *Mathematica* using large collection of rules with `/;` conditions.